

Latihan Soal Graph

IF2110/IF2111 – Algoritma dan Struktur Data
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Latihan Graph

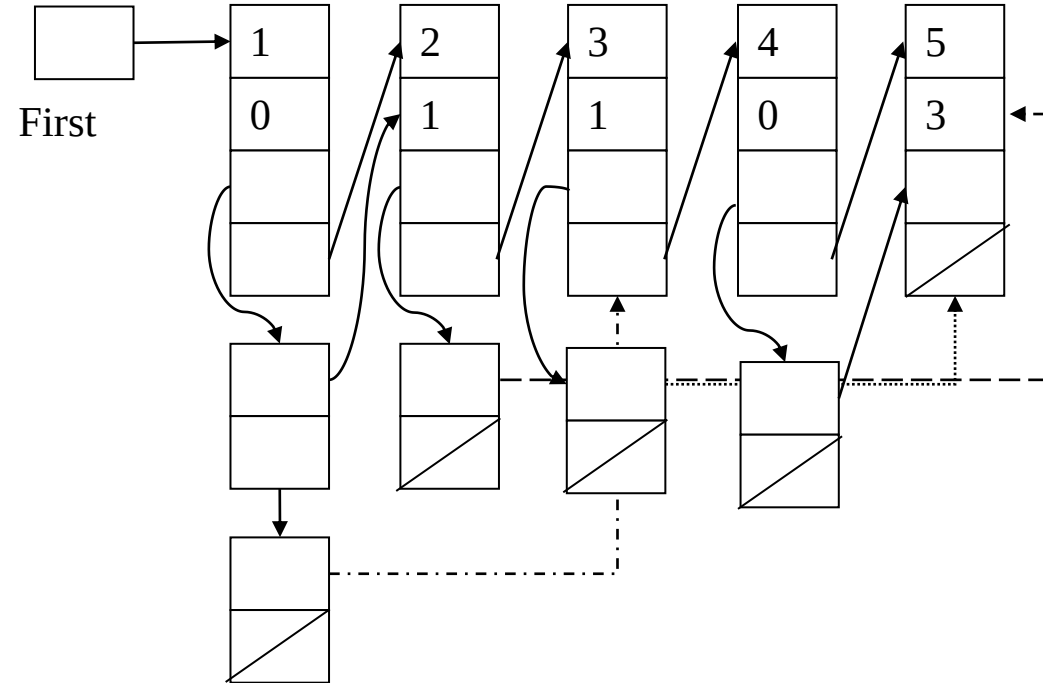
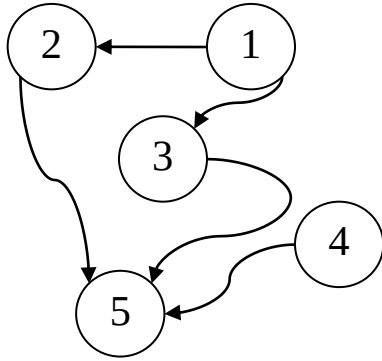
Graph Berarah adalah graph dengan busur yang memiliki arah. Artinya, jika pada graph terdefinisi busur (a, k) berarti ada busur dengan titik awal simpul a (*predecessor*) menuju simpul k (*successor*).

Graph Berarah akan diimplementasikan dengan menggunakan variasi *adjacency list*, dalam bentuk multilist berikut:

List simpul (*leader list*) adalah list dengan elemen seluruh simpul yang terdapat di dalam graph. Setiap elemen list berisi identitas simpul (id), jumlah busur yang masuk ke simpul tersebut (npred), pointer ke list simpul *successornya* (trail), dan pointer ke elemen simpul berikutnya (next). Implementasi dalam bentuk list dengan pencatatan First, dengan elemen terurut membesar berdasarkan atribut id.

List successor (*trailer list*) adalah list yang berisi pointer ke simpul-simpul yang menjadi successor dari simpul utamanya.

Latihan Graph



```
{ Graph berarah diimplementasi sebagai Multilist }  
constant NIL: ...  
type AdrNode: pointer to Node  
type AdrSuccNode: pointer to SuccNode  
type Node: < id: integer,    {identitas simpul}  
            nPred: integer,    {jumlah busur yang masuk ke simpul =  
                                jumlah simpul pred}  
            trail: AdrSuccNode, {pointer ke list trailer (simpul successor)}  
            next: AdrNode >  
type SuccNode: < succ: AdrNode,    {address simpul successor}  
                next: AdrSuccNode >  
type Graph: < first: AdrNode >
```

```
{ *** Konstruktor *** }  
procedure CreateGraph (input x: integer, output g: Graph)  
{ I.S. Sembarang; F.S. Terbentuk Graph dengan satu simpul dengan Id=X }
```

```
{ *** Manajemen Memory List Simpul (Leader) *** }  
function newGraphNode(x: integer) → AddrNode  
{ Mengembalikan address hasil alokasi Simpul x. }  
{ Jika alokasi berhasil, maka address tidak NIL, misalnya  
menghasilkan p, maka p↑.id=x, p↑.nPred=0, p↑.trail=NIL,  
dan p↑.next=NIL. Jika alokasi gagal, mengembalikan NIL. }
```

```
procedure deallocGraphNode (input p: AddrNode)  
{ I.S. P terdefinisi; F.S. P dikembalikan ke sistem }
```

```
{ *** Manajemen Memory List Successor (Trailer) *** }  
function newSuccNode (pn: AddrNode) → AddrSuccNode  
{ Mengembalikan address hasil alokasi. }  
{ Jika alokasi berhasil, maka address tidak NIL, misalnya  
menghasilkan pt, maka pt↑.succ=pn dan pt↑.next=NIL. Jika  
alokasi gagal, mengembalikan NIL. }
```

```
procedure deallocSuccNode (input p: AddrSuccNode)  
{ I.S. p terdefinisi; F.S. p dikembalikan ke sistem }
```

```
{ *** Fungsi/Prosedur Lain *** }
```

```
function searchNode (g: Graph, x: integer) → AdrNode
```

```
{ mengembalikan address simpul dengan id=x jika sudah ada pada graph g,  
  NIL jika belum }
```

```
function searchEdge (g: Graph, prec: integer, succ: integer) → adrSuccNode
```

```
{ mengembalikan address trailer yang menyimpan info busur <prec,succ>  
  jika sudah ada pada graph g, NIL jika belum }
```

```
procedure insertNode (input/output g: Graph, input x: integer, output pn: adrNode)
```

```
{ Menambahkan simpul x ke dalam graph g, jika alokasi x berhasil. }  
{ I.S. g terdefinisi, x terdefinisi dan belum ada pada g. }  
{ F.S. Jika alokasi berhasil, x menjadi elemen terakhir g, pn berisi  
  address simpul x. Jika alokasi gagal, g tetap, pn berisi NIL }
```

```
procedure insertEdge (input/output g: Graph, input prec, succ: integer)
```

```
{ Menambahkan busur dari prec menuju succ ke dalam G }  
{ I.S. g, prec, succ terdefinisi. }  
{ F.S. Jika belum ada busur <prec,succ> di g, maka tambahkan busur  
  <prec,succ> ke g. Jika simpul prec/succ belum ada pada g,  
  tambahkan simpul tersebut dahulu. Jika sudah ada busur <prec,succ>  
  di g, maka g tetap. }
```

```
procedure deleteNode (input/output g: Graph, input x: integer)  
{ Menghapus simpul x dari g }  
{ I.S. g terdefinisi, x terdefinisi dan ada pada g, jumlah simpul  
  pada g lebih dari 1. }  
{ F.S. simpul x dan semua busur yang terhubung ke x dihapus  
  dari g. }
```

Implementasikan primitif **searchNode**, **searchEdge**, **insertNode**, **insertEdge**, dan **deleteNode** yang ada sesuai definisi dan spesifikasi di atas, tanpa menulis ulang spesifikasinya. Tidak boleh membuat fungsi/prosedur baru.